

W14D06 — Project 3 Backend dengan Database

ML Engineer Journey · Phase 4: Database & Observability · 13 Jun 2026

1. Overview & Tujuan Sesi

Hari ini adalah puncak dari Week 14. Lima hari sebelumnya dihabiskan untuk mempelajari fondasi: ORM, migration, async SQLAlchemy, dan connection pooling. Hari ini semua fondasi tersebut dirakit menjadi satu sistem backend yang utuh — production-grade, berlapis, dan end-to-end test berjalan.

Goal konkret sesi ini: **semua endpoint Project 3 sudah tersimpan dan membaca dari database, end-to-end test berjalan**. Bukan sekadar kode yang 'works in isolation' — tapi sistem yang bisa diulang, dites, dan didokumentasikan.

2. Arsitektur: Separation of Concerns

Bayangkan sebuah restoran. Ada **pelayan** yang terima pesanan, **koki** yang masak, **gudang** yang kelola bahan baku, dan **database resep**. Pelayan tidak masak. Koki tidak kelola gudang. Setiap orang tahu tugasnya — dan hanya tugasnya.

Itulah prinsip Separation of Concerns yang menjadi tulang punggung arsitektur ini. Setiap layer memiliki satu tanggung jawab yang jelas, dan tidak boleh 'mencuri pekerjaan' layer lain. Ini bukan formalitas — ini yang membuat codebase bisa di-maintain dan di-test.

Struktur Folder Production-Grade

Path	Tanggung Jawab
main.py	Entry point: lifespan, app init, router register
app/config.py	Pydantic Settings — baca .env, validasi tipe otomatis
app/exceptions.py	Custom HTTPException subclass — satu tempat ubah format error
app/database.py	Async engine + get_db() generator (dari W14D05)
app/models/	SQLAlchemy ORM model — definisi tabel
app/schemas/	Pydantic Request/Response schema — validasi I/O
app/repositories/	Semua query DB — CRUD, filter, search
app/services/	Business logic — validasi domain, orchestrate repo calls
app/routers/	HTTP routing — terima request, delegasi ke service, kembalikan re
tests/	test_routers.py, test_services.py, test_repositories.py
alembic/	Database migrations (dari W14D03)

3. Tanggung Jawab Tiap Layer

Layer	Boleh	TIDAK Boleh
Router	HTTP routing, response format, status code	Business logic, query DB langsung
Service	Business rules, validasi domain, raise NotFoundException	Query DB langsung, return HTTP response
Repository	Query DB, CRUD, filter, search	Business rules, raise HTTPException
Model	Definisi tabel ORM, relationship	Business logic apapun
Schema	Validasi I/O Pydantic, type hints	ORM, DB logic

Request flow yang benar: HTTP Request → Router → Service → Repository → DB → Repository → Service → Router → HTTP Response. Tidak ada shortcut. Tidak ada layer yang melompati layer lain.

4. Implementasi Kode — Layer by Layer

4.1 config.py — Pydantic Settings

Pydantic Settings otomatis membaca environment variables dan file .env, sekaligus memvalidasi tipe datanya. Jauh lebih aman dibanding os.getenv() yang bisa mengembalikan None tanpa peringatan.

```
# app/config.py
from pydantic_settings import BaseSettings # pip install pydantic-settings

class Settings(BaseSettings):
    database_url: str # wajib ada di .env – error jika tidak ada
    app_name: str = 'Project 3 API'
    debug: bool = False

class Config:
    env_file = '.env' # baca otomatis dari .env

settings = Settings() # singleton – import dari mana saja
```

4.2 exceptions.py — Custom HTTP Exceptions

Daripada menulis raise HTTPException(404, 'not found') di 20 endpoint berbeda, buat satu class yang bisa dipakai ulang. Kalau format error berubah, cukup ubah di satu tempat.

```
# app/exceptions.py
from fastapi import HTTPException, status

class NotFoundException(HTTPException):
    def __init__(self, resource: str, id: str):
        super().__init__(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"{resource} with id '{id}' not found"
        )

class ValidationException(HTTPException):
```

```

def __init__(self, message: str):
    super().__init__(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        detail=message
    )

```

4.3 repositories/ — Semua Query DB

Repository adalah satu-satunya layer yang boleh menyentuh database. Tidak ada business rule di sini. Tidak ada HTTP exception. Hanya query.

```

# app/repositories/analysis_repo.py
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.models.analysis import Analysis

class AnalysisRepository:
    @staticmethod
    async def get_all(db: AsyncSession) -> list[Analysis]:
        result = await db.execute(select(Analysis)) ## async – bukan db.query()
        return result.scalars().all() ## .scalars() unwrap ORM objects

    @staticmethod
    async def get_by_id(db: AsyncSession, analysis_id: str) -> Analysis | None:
        return await db.get(Analysis, analysis_id) ## cek identity map dulu (efisien)

    @staticmethod
    async def create(db: AsyncSession, data: dict) -> Analysis:
        obj = Analysis(**data)
        db.add(obj)
        await db.flush() ## generate UUID di Postgres TANPA commit
        await db.refresh(obj) ## sync state object dengan DB
        return obj

    @staticmethod
    async def update(db: AsyncSession, obj: Analysis, data: dict) -> Analysis:
        for key, value in data.items():
            setattr(obj, key, value) ## update hanya field yang dikirim
        await db.flush()
        await db.refresh(obj)
        return obj

    @staticmethod
    async def delete(db: AsyncSession, obj: Analysis) -> None:
        await db.delete(obj)
        await db.flush()

```

4.4 services/ — Business Logic

Service tahu tentang business rules, tahu tentang exception 404, tapi tidak tahu soal HTTP response format — itu urusan router.

```
# app/services/analysis_service.py
from sqlalchemy.ext.asyncio import AsyncSession

from app.repositories.analysis_repo import AnalysisRepository
from app.schemas.analysis import AnalysisCreate, AnalysisUpdate
from app.exceptions import NotFoundException, ValidationException

class AnalysisService:

    @staticmethod
    async def get_by_id(db: AsyncSession, analysis_id: str):
        obj = await AnalysisRepository.get_by_id(db, analysis_id)
        if not obj:
            raise NotFoundException('Analysis', analysis_id) ## business rule: 404
        return obj

    @staticmethod
    async def create(db: AsyncSession, payload: AnalysisCreate):
        if payload.score < 0 or payload.score > 100:
            raise ValidationException('Score harus antara 0 dan 100')
        return await AnalysisRepository.create(db, payload.model_dump())

    @staticmethod
    async def update(db: AsyncSession, analysis_id: str, payload: AnalysisUpdate):
        obj = await AnalysisService.get_by_id(db, analysis_id) ## validasi dulu
        return await AnalysisRepository.update(
            db, obj, payload.model_dump(exclude_unset=True) ## hanya field yg dikirim
        )
```

4.5 routers/ — HTTP Endpoints

Router hanya bertanggung jawab pada: terima request, delegasi ke service, kembalikan response dengan status code yang tepat. Tidak ada query DB. Tidak ada if not found.

```
# app/routers/analysis.py
from fastapi import APIRouter, Depends, status

from sqlalchemy.ext.asyncio import AsyncSession

from app.database import get_db
from app.services.analysis_service import AnalysisService
from app.schemas.analysis import AnalysisCreate, AnalysisUpdate, AnalysisResponse

router = APIRouter(prefix='/analyses', tags=['Analysis'])

@router.get('/', response_model=list[AnalysisResponse])
async def list_analyses(db: AsyncSession = Depends(get_db)):
    return await AnalysisService.get_all(db)
```

```

@router.post('/', response_model=AnalysisResponse, status_code=status.HTTP_201_CREATED)
async def create_analysis(payload: AnalysisCreate, db: AsyncSession = Depends(get_db)):
    return await AnalysisService.create(db, payload)

@router.patch('/{analysis_id}', response_model=AnalysisResponse)
async def update_analysis(
    analysis_id: str, payload: AnalysisUpdate, db: AsyncSession = Depends(get_db)
):
    return await AnalysisService.update(db, analysis_id, payload)

@router.delete('/{analysis_id}', status_code=status.HTTP_204_NO_CONTENT)
async def delete_analysis(analysis_id: str, db: AsyncSession = Depends(get_db)):
    await AnalysisService.delete(db, analysis_id)

```

5. Konsep Kritis yang Harus Dipahami

5.1 flush() vs commit() vs refresh()

Tiga operasi ini sering membingungkan. Berikut perbedaan fundamentalnya:

Operasi	Apa yang Terjadi	Kapan Dipakai
flush()	Kirim SQL ke DB dalam transaksi aktif — BELUM commit. Setelah flush(), setiap perubahan DB	Setelah flush(), setiap perubahan DB sert child yang butuh
commit()	Finalisasi transaksi — perubahan permanen di DB.	Di get_db() setelah request sukses
refresh()	Sync state Python object dengan nilai aktual di DB (UUID, Setelah flush() Setelah commit()).	untuk mendapatkan nilai yang di-generat

Urutan wajib: flush() → refresh(). Tidak bisa dibalik. Kalau refresh() dipanggil sebelum flush(), Postgres belum generate UUID — refresh() tidak punya data untuk di-sync, dan akan error.

5.2 Kenapa update() harus get_by_id() dulu?

Ada dua alasan fundamental mengapa service.update() tidak boleh langsung panggil repository.update() dengan ID tanpa mengambil objeknya terlebih dahulu:

- **Validasi keberadaan data:** Jika langsung UPDATE WHERE id=x tanpa cek, query akan 'sukses' (mengubah 0 baris) meski datanya tidak ada. Aplikasi tidak tahu apakah itu berhasil mengubah data atau datanya memang kosong.
- **Cara kerja ORM:** SQLAlchemy bekerja dengan object tracking — bukan raw SQL. Cara paling natural dan aman adalah: ambil objek, ubah atributnya, biarkan SQLAlchemy generate UPDATE statement sendiri berdasarkan perubahan yang ia deteksi.

5.3 model_dump(exclude_unset=True) untuk PATCH

PATCH dalam standar REST berarti 'ubah hanya sebagian field'. Ini adalah perbedaan kritis antara PATCH dan PUT.

Method	Semantik	Harus Kirim	Dampak jika field tidak ada
PUT	Replace seluruh resource	Semua field	Field yang tidak dikirim menjadi null/default

PATCH	Update sebagian field saja	Hanya field yang berubah	Field yang tidak dikirim TIDAK BOLEH berubah
-------	----------------------------	--------------------------	--

Tanpa `exclude_unset=True`, Pydantic akan mengisi semua field yang tidak dikirim client dengan nilai `None`. Saat dict ini dikirim ke repository, ORM akan menimpa semua kolom termasuk yang seharusnya tidak berubah — data pengguna hilang tanpa izin.

6. Knowledge Check — Jawaban Tervalidasi

Pertanyaan	Jawaban
Kenapa <code>flush()</code> harus sebelum <code>refresh()</code> ?	<code>flush()</code> mengirim SQL INSERT ke DB dalam transaksi aktif — ini yang men-trigger Postgres untuk flush ke disk.
Kenapa <code>service.update()</code> harus <code>get_by_id()</code> dulu?	Dipastikan: (1) Validasi keberadaan — UPDATE tanpa cek akan 'sukses' mengubah 0 baris jika ID tidak ada.
Apa yang terjadi jika PATCH pakai <code>model_dump()</code> tanpa <code>exclude_unset=True</code> ?	<code>Pydantic</code> mengisi <code>exclude_unset=True</code> tidak dikirim client dengan <code>None</code> . Saat dict ini dikirim ke repository, ORM akan menimpa semua kolom termasuk yang seharusnya tidak berubah — data pengguna hilang tanpa izin.

7. Checklist End-to-End

- `uvicorn main:app --reload` berjalan tanpa error
- Semua 6 endpoint merespons dengan benar (GET list, GET by ID, POST, PATCH, DELETE, health)
- 404 muncul saat resource tidak ada — bukan 500 Internal Server Error
- PATCH hanya update field yang dikirim, tidak overwrite semua field
- Tidak ada `db.query()` di manapun — semua sudah await `db.execute(select(...))`
- Tidak ada business logic di router layer
- Tidak ada `HTTPException` di repository layer
- 2 test berjalan: `test_create_and_get_prediction` PASSED, `test_create_prediction_validation_error` PASSED

8. Quick Reference — Async SQLAlchemy Pattern

```
## GET ALL
result = await db.execute(select(Model))
items = result.scalars().all()

## GET BY PK (efisien – cek identity map dulu)
obj = await db.get(Model, pk)

## CREATE
obj = Model(**data)
db.add(obj)
await db.flush() ## generate UUID/defaults di Postgres
await db.refresh(obj) ## sync Python object dengan nilai aktual DB

## UPDATE (via object tracking)
for key, val in patch_data.items():
    setattr(obj, key, val)
await db.flush()
await db.refresh(obj)
```

```
## DELETE
await db.delete(obj)
await db.flush()

## SESSION (dari get_db – handle commit/rollback/close otomatis)
async def get_db():
    async with AsyncSessionLocal() as db:
        try:
            yield db
        except:
            await db.rollback()
            raise
```

W14D06 — Project 3 Backend dengan Database · ML Engineer Journey Phase 4 · 13 Jun 2026

Next: W15 — Observability, Logging & Monitoring